SnapRAG

Verified Reconstruction of Snapshot Memory for Long-Conversation Document QA

Funding request for API-based benchmarking and evaluation

Prepared by	[Your name], [Institution / department]
Supervisor / sponsor	[Name]
Date	[Date]
Amount requested	**USD 300** (recommended plan). Lean plan USD 150, full plan USD 790. [INR equivalent at current rate]
Purpose	Pay for paid LLM API usage to run the benchmarks that decide whether the research claims hold

1. Summary

SnapRAG is a working question-answering system for long conversations about documents. When a conversation grows beyond what a model can read, older turns are kept in two forms: the full text in an immutable archive, and a small lossy snapshot. When old information is needed again, the system rebuilds text from the snapshot and **checks every sentence against the snapshot, the document pages and the archive** before the model may use it. Sentences that cannot be checked are dropped.

The system is built and tested (175 automated tests pass). What is missing is evidence. The benchmark that produces that evidence needs about 85 million input tokens and 8 million output tokens of LLM usage. The free Gemini tier we have used so far is too slow and too rate-limited for this (a single turn took 50 to 135 seconds in recent runs), and a free tier also cannot give the repeatable, citable results a paper needs.

Request: USD 300 of paid API credit. This covers a pilot, the full 39-run benchmark, a held-out calibration run, a second document, a second LLM vendor, and a 30% contingency for re-runs.

Result: a measured answer to whether verified memory reduces forgotten and invented facts, at what token and cost saving, plus a public benchmark (SnapRAG-LongConv) that others can reuse.

2. Problem and idea

Chat assistants over documents (study material, legal text, manuals) lose information in long sessions. Today there are three common fixes, and each has a weakness:

- **Bigger context window:** cost grows with every turn and models still miss facts buried in the middle.
- **Summaries:** cheap, but the loss is silent. The model answers from a distorted summary with full confidence.
- **Memory stores:** retrieve past facts, but do not check that the retrieved text is true.

SnapRAG's idea is: *compress, reconstruct later, check against evidence, only then use.*

2.1 What is built

Component	What it does
Cold archive	Every turn kept as append-only text. Never edited, so it is the final authority.
Hot snapshots	Small hash-chained summaries of facts, numbers, user statements, decisions and open questions with document/page anchors.
Verified reconstruction	Rebuilds text from a snapshot, checks each sentence against snapshot, pages and archive, drops what fails.
Quality trigger (CIS)	A score of grounding, coherence, contradiction and headroom decides when to snapshot.
Conversation state	User decisions and statements stored as versioned items, each verified against the user's own words.
Adaptive retrieval (AB-RAG based)	Reads more passages only while answer confidence is low; budget depends on question type; verified memory counts as evidence.
Benchmark SnapRAG-LongConv	100-turn scripted conversation over a real torts textbook with 24 memory probes at four distances.

2.2 Honest position on novelty

Hybrid retrieval, hash-chained versions and confidence-gated retrieval are known techniques (the last one is the published AB-RAG idea and we cite it). The claimed contributions are: **(C1)** checking reconstructed memory sentence by sentence against the archive and pages, **(C2)** the snapshot trigger and turn-aware memory weight, **(C3)** a long document-QA conversation benchmark, and **(C4)** intent- and memory-aware retrieval budgets. C1 is the main claim. A full literature check is still pending, so we claim moderate novelty only and will not write "first" or "most accurate" in the paper.

2.3 Early evidence (not final)

An unfinished run (80-turn script, 74 turns done, free tier) answered 16 of 21 probes correctly (76%). All "not told yet" probes were declined correctly, and a correction probe passed. Five probes failed: one trap probe where the system accepted a false premise and four recalls of far-away facts. One run on 21 probes has wide uncertainty and is **not** a result. It shows the system works and that failures exist worth studying.

3. What will be tested

Each claim has a matching experiment. Results are reported with confidence intervals from 3 repeated runs, and as paired differences between variants on the same script.

#	Claim	Experiment	Variants compared	Measured
T1	Memory helps	LongConv main	full vs no_memory, full_history, summary_buffer	Probe accuracy by distance, memory loss, tokens per turn
T2	C1: verification matters	Ladder and reconstruction eval	snapshot, evidence, verify, archive	Unsupported claims, fact recall
T3	C2: trigger and weight	Ablation	full vs fixed_interval, fixed_lambda	Accuracy, snapshots made, tokens
T4	C4: adaptive retrieval	Ablation	full vs fixed_k (no extra batches)	Accuracy, page-hit, passages read, LLM calls
T5	Single-turn quality	Retrieval and grounding eval	legal question set	Citation correctness, grounding
T6	No tuning on test data	Held-out calibration	New script, new seed	Best AB-RAG threshold and weights
T7	Scoring is valid	Manual check	50 answers read by hand	Agreement with the regex scorer
T8	Generalises	Second document and second LLM vendor	full vs no_memory	Same metrics as T1

Scoring uses fixed rules (regular expressions), not another LLM, so there is no judge cost and no judge bias. An automated dry-run with a perfect oracle checks the scorer itself.

3.1 Why paid API access is needed

- **Speed:** free-tier latency reached 50 to 135 seconds per turn and one turn took 256 seconds. At about 100 turns per run and 39 runs, that is unusable. Paid access runs several runs in parallel.
- **Repeatability:** reviewers expect a fixed, named model version and stable limits. Free tiers change and throttle.
- **Quality of the experiment:** verification and reconstruction depend on strict JSON and instruction following, which stronger models do more reliably.
- **Second vendor:** a typical review comment is "only one LLM was tested". One extra backbone answers it.

4. Model selection

The code talks to an OpenAI-style gateway, so any provider can be placed behind it. The plan uses a strong model for answering and checking, and a cheap model for bulk work (planner, snapshot upkeep). Prices are per million tokens (list prices cached on 2026-10-06; confirm on the provider's pricing page before paying).

Role	Model	Input	Output	Why
Main (answer, check)	Claude Sonnet 5.5	$2.00	$10.00	Strict-format reliability at a middle price
Cheap (planner, upkeep)	Claude Haiku 5.5	$0.10	$0.50	Handles the bulk of simple calls
Premium option	Claude Opus 5.5	$4.00	$20.00	Only if the budget allows a top-model run
Second vendor	One paid GPT or Gemini model	check	check	Generalisation across LLM families

Disclosure: this proposal was prepared with help from Claude, which is one of the candidate models. The second-vendor run and the fixed, rule-based scorer are there so that the conclusions do not depend on one vendor.

5. Cost estimate

5.1 Assumptions

- Measured from our logs: a turn uses about 5 to 6 chat calls plus 0 to 4 memory-upkeep calls.
- **Assumed, not yet measured:** about 16,000 input and 2,000 output tokens per turn. A pilot will replace this guess.
- A 100-turn run: about 1.8M input and 0.2M output tokens. The full-history baseline sends the whole chat each turn: about 6M input per run.
- Embeddings and the re-ranker run locally, so they cost nothing. No LLM judge is used.
- Estimates may be off by roughly 50% in either direction.

5.2 Token volume of the full plan

Item	Runs	Input tokens	Output tokens
Memory variants (full, ladder, ablations, no_memory, summary_buffer, state_only, fixed_k)	36	about 65M	about 7M
Full-history baseline	3	about 18M	about 1M
Total, 13 variants x 3 repeats	**39**	**about 83M**	**about 8M**

5.3 Price of the main benchmark by model choice

Setup	Calculation	Main benchmark
All Haiku 5.5	83M x $0.10 + 8M x $0.50	about $12
Sonnet 5.5 for answers and checks, Haiku 5.5 for bulk (recommended)	mixed	about $100 - $150
All Sonnet 5.5	83M x $2 + 8M x $10	about $246
All Opus 5.5	83M x $4 + 8M x $20	about $492
All Fable 5.1 (not advised)	83M x $10 + 8M x $50	about $1,230

5.4 Requested budget (three plans)

Line item	Lean	Recommended	Full
Pilot run (1 run, token measurement)	$5	$5	$10
Main benchmark, 13 variants	$100 (2 repeats)	$150 (3 repeats)	$492 (all Opus)
Held-out calibration and 2nd document (about 10 runs)	$25	$40	$60
Second LLM vendor (6 runs, full and no_memory)	not included	$40 (estimate)	$40 (estimate)
Subtotal	$130	$235	$602
Contingency (re-runs after bug fixes)	15%: $20	30%: $70	30%: $181
Total requested	**$150**	**$305, rounded to $300**	**$783, rounded to $790**

The recommended plan is the smallest one that supports every claim in the paper, including the second-vendor check. The lean plan can only support T1 to T4 with weaker intervals. The full plan is only worth it if a top model is specifically required.

5.5 Cost controls

- A 1-run pilot first (about $5). Real token counts replace the assumption and the total is re-estimated before the rest is spent.
- Token usage will be logged for every run (added in week 1), so the spend can be checked against this table at any time.
- A hard spending limit will be set on the API account at the requested amount.
- Runs resume from saved results, so an interruption does not repeat paid work.
- Cheap model for planner and upkeep calls; prompt caching where the provider supports it.

6. Plan and timeline

Week	Work	Output
1	Add token logging and provider switch; pilot run; re-estimate cost; fix bugs the pilot shows (for example the false-premise miss)	Confirmed budget, stable code
2	Main benchmark: baselines and full system, 3 repeats	Table 3 of the paper (main results)
3	Ladder, CIS/lambda ablations, fixed-k ablation	Ablation tables, accuracy-vs-distance figure
4	Held-out calibration, manual scorer check, second document, second vendor	Calibrated settings, validity evidence
5	Literature check, analysis of failure cases, write results	Results and related-work sections
6	Write remaining sections, release code and benchmark	10-page paper draft, public repository

Runs take roughly one hour each over a paid API and can be run in parallel, subject to rate limits. Sequentially, 39 runs would take about 40 hours.

7. Deliverables

- A 10-page research paper with measured results and confidence intervals.
- The SnapRAG-LongConv benchmark: script generator, scorer and dry-run oracle, openly released.
- Open-source code with 175+ automated tests and a continuous-integration check.
- Raw result files for every run, so every number in the paper can be reproduced.

8. Risks and how they are handled

Risk	Handling
Token use is higher than assumed	Pilot first; contingency; spending cap on the account
Results do not favour the system	Report them honestly; the failure analysis is still a contribution and the benchmark stays useful
Novelty already exists in papers we have not read	Literature search before writing claims; claims limited to what is shown
Single document and single domain	Second document in plan; stated as a limitation
Scorer disagrees with human judgement	Manual check of 50 answers; report agreement
AB-RAG weights tuned on the test script	Calibrate on a separate held-out script with another seed

9. Current status

Item	Status
Core system (memory, verification, state, adaptive retrieval)	Built and tested
Automated tests	175 passing; static check clean
Benchmark generator, scorer, variants, dry-run	Built
Real-LLM benchmark runs	One partial free-tier run only; full runs wait for this funding

Item	Status
Literature check for novelty	Pending
Paper	Template and plan ready; results blank until measured

10. Request

I request **USD 300** (about [INR amount]) of paid API credit, to be used only for the benchmark runs described above, with the pilot spend reported to you before the remainder is used. I will share the usage log and the raw results when the work is complete.

[Your name] [Signature / date] Approved by: [Name]